UNIT-I — Sorting

1. Summation of 1st N Natural Numbers — 3 Methods

Method 1: Using Loop

```
n = int(input("Enter n: "))

sum = 0
for i in range(1, n + 1):
    sum += i

print("Sum =", sum)
```

Method 2: Using Formula

```
n = int(input("Enter n: "))

sum = n * (n + 1) // 2

print("Sum =", sum)
```

Method 3: Using Recursion

```
def summation(n):
    if n == 0:
        return 0
    return n + summation(n - 1)

n = int(input("Enter n: "))
print("Sum =", summation(n))
```

Example: `n = 5` → `1+2+3+4+5 = 15`

---

2. Prime Number — 3 Methods

Method 1: Simple Loop

```
n = int(input("Enter number: "))
```

```
if n < 2:
    print("Not Prime")
else:
    count = 0

    for i in range(1, n + 1):
        if n % i == 0:
            count += 1

    if count == 2:
        print("Prime")
    else:
        print("Not Prime")
```

Method 2: Using `break`

```
n = int(input("Enter number: "))

if n < 2:
    print("Not Prime")
else:
    prime = True

    for i in range(2, n):
        if n % i == 0:
            prime = False
            break

    if prime:
        print("Prime")
    else:
        print("Not Prime")
```

Method 3: Check up to √n

```
import math

n = int(input("Enter number: "))

if n < 2:
    print("Not Prime")
else:
    prime = True

    for i in range(2, int(math.sqrt(n)) + 1):
```

```
        if n % i == 0:
            prime = False
            break

    if prime:
        print("Prime")
    else:
        print("Not Prime")
```

Easy trick: Prime number has only **2 factors: 1 and itself**.

---

3. Fibonacci — Iterative and Recursive

Iterative Method

```
n = int(input("Enter number of terms: "))

a = 0
b = 1

for i in range(n):
    print(a, end=" ")
    a, b = b, a + b
```

Output for n=7:

0 1 1 2 3 5 8

Recursive Method

```
def fibonacci(n):
    if n <= 1:
        return n

    return fibonacci(n - 1) + fibonacci(n - 2)

n = int(input("Enter number of terms: "))

for i in range(n):
    print(fibonacci(i), end=" ")
```

---

4. Bubble Sort

Logic: Compare adjacent elements and swap if they are in wrong order.

```
arr = [5, 3, 8, 2, 7]

n = len(arr)

for i in range(n):
    for j in range(0, n - i - 1):
        if arr[j] > arr[j + 1]:
            arr[j], arr[j + 1] = arr[j + 1], arr[j]

print("Sorted array:", arr)
```

Output:

```
[2, 3, 5, 7, 8]
```

genui{"learning_viz":{"type_id":"INSERTION_SORT","locale_override":"en-US"}}

Complexity:
Best = O(n)
Average = O(n²)
Worst = O(n²)

---

5. Insertion Sort

Logic: Take one element and insert it into its correct position in the sorted part.

```
arr = [5, 3, 8, 2, 7]

for i in range(1, len(arr)):
    key = arr[i]
    j = i - 1

    while j >= 0 and arr[j] > key:
        arr[j + 1] = arr[j]
        j -= 1

    arr[j + 1] = key
```

```
print("Sorted array:", arr)
```

Output:

[2, 3, 5, 7, 8]

Complexity:
Best = O(n)
Average = O(n²)
Worst = O(n²)

---

6. Selection Sort

Logic: Find the smallest element and put it at the beginning.

```
arr = [5, 3, 8, 2, 7]

n = len(arr)

for i in range(n):
    min_index = i

    for j in range(i + 1, n):
        if arr[j] < arr[min_index]:
            min_index = j

    arr[i], arr[min_index] = arr[min_index], arr[i]

print("Sorted array:", arr)
```

Output:

[2, 3, 5, 7, 8]

Complexity:
Best = O(n²)
Average = O(n²)
Worst = O(n²)

genui{"learning_viz":{"type_id":"SELECTION_SORT","locale_override":"en-US"}}

UNIT-II — Divide and Conquer

1. Binary Search

Important: Array must be **sorted**.

```
def binary_search(arr, key):
    low = 0
    high = len(arr) - 1

    while low <= high:
        mid = (low + high) // 2

        if arr[mid] == key:
            return mid

        elif arr[mid] < key:
            low = mid + 1

        else:
            high = mid - 1

    return -1


arr = [10, 20, 30, 40, 50]
key = int(input("Enter element: "))

result = binary_search(arr, key)

if result != -1:
    print("Element found at index", result)
else:
    print("Element not found")
```

Complexity: O(log n)

2. Merge Sort

Logic: Divide array → sort each part → merge.

```
def merge_sort(arr):

    if len(arr) <= 1:
        return arr

    mid = len(arr) // 2

    left = merge_sort(arr[:mid])
    right = merge_sort(arr[mid:])

    result = []
    i = 0
    j = 0

    while i < len(left) and j < len(right):

        if left[i] < right[j]:
            result.append(left[i])
            i += 1
        else:
            result.append(right[j])
            j += 1

    result.extend(left[i:])
    result.extend(right[j:])

    return result


arr = [8, 3, 5, 2, 7, 1]

print("Sorted array:", merge_sort(arr))
```

Complexity: O(n log n)

genui{"learning_viz":{"type_id":"MERGE_SORT","initial_values":{"value1":8,"value2":3,"value3":5,"value4":2,"value5":7,"value6":1,"value7":6,"value8":4},"locale_override":"en-US"}}

---

3. Quick Sort

Logic: Select pivot → smaller elements left → larger elements right.

```
def quick_sort(arr):

    if len(arr) <= 1:
        return arr

    pivot = arr[0]

    left = [x for x in arr[1:] if x <= pivot]
    right = [x for x in arr[1:] if x > pivot]

    return quick_sort(left) + [pivot] + quick_sort(right)


arr = [5, 3, 8, 2, 7, 1]

print("Sorted array:", quick_sort(arr))
```

Output:

```
[1, 2, 3, 5, 7, 8]
```

Complexity:
Best/Average = O(n log n)
Worst = O(n²)

---

4. Large Number Multiplication — Karatsuba

For practical, this simple recursive code is easy to remember:

```
def karatsuba(x, y):

    if x < 10 or y < 10:
        return x * y

    n = max(len(str(x)), len(str(y)))
```

```
    m = n // 2

    high1 = x // (10 ** m)
    low1 = x % (10 ** m)

    high2 = y // (10 ** m)
    low2 = y % (10 ** m)

    z0 = karatsuba(low1, low2)
    z1 = karatsuba(low1 + high1, low2 + high2)
    z2 = karatsuba(high1, high2)

    return z2 * (10 ** (2 * m)) + \
           (z1 - z2 - z0) * (10 ** m) + z0


x = int(input("Enter first number: "))
y = int(input("Enter second number: "))

print("Multiplication =", karatsuba(x, y))
```

Main idea:

z0 = low × low
z1 = (low + high) × (low + high)
z2 = high × high

Complexity: $O(n^{\log_2 3}) \approx O(n^{1.585})$

---

5. Strassen's Matrix Multiplication

For **2 × 2 matrices**:

```
A = [[1, 2],
     [3, 4]]

B = [[5, 6],
     [7, 8]]

a, b = A[0]
c, d = A[1]
```

```
e, f = B[0]
g, h = B[1]

M1 = (a + d) * (e + h)
M2 = (c + d) * e
M3 = a * (f - h)
M4 = d * (g - e)
M5 = (a + b) * h
M6 = (c - a) * (e + f)
M7 = (b - d) * (g + h)

C11 = M1 + M4 - M5 + M7
C12 = M3 + M5
C21 = M2 + M4
C22 = M1 - M2 + M3 + M6

C = [[C11, C12],
     [C21, C22]]

print("Result:")
for row in C:
    print(row)
```

Output:

```
[19, 22]
[43, 50]
```

Complexity: O(n^2.807)

---

6. Find Minimum and Maximum

Divide and Conquer Method

```
def min_max(arr, low, high):

    if low == high:
        return arr[low], arr[low]

    if high == low + 1:
        if arr[low] < arr[high]:
```

```
            return arr[low], arr[high]
        else:
            return arr[high], arr[low]

    mid = (low + high) // 2

    min1, max1 = min_max(arr, low, mid)
    min2, max2 = min_max(arr, mid + 1, high)

    minimum = min(min1, min2)
    maximum = max(max1, max2)

    return minimum, maximum


arr = [5, 2, 9, 1, 7, 6]

minimum, maximum = min_max(arr, 0, len(arr) - 1)

print("Minimum =", minimum)
print("Maximum =", maximum)
```

Output:

```
Minimum = 1
Maximum = 9
```

---

7. Maximum Subarray

This is commonly solved using **Divide and Conquer**.

```
def max_subarray(arr):

    max_sum = arr[0]
    current_sum = arr[0]

    for i in range(1, len(arr)):

        current_sum = max(arr[i], current_sum + arr[i])

        max_sum = max(max_sum, current_sum)
```

```
    return max_sum


arr = [-2, 1, -3, 4, -1, 2, 1, -5, 4]

print("Maximum Subarray Sum =", max_subarray(arr))
```

Output:

Maximum Subarray Sum = 6

Because:

4 + (-1) + 2 + 1 = 6

UNIT-III — Dynamic Programming

1. Coin Change

Problem: Find minimum number of coins needed to make a given amount.

```
def coin_change(coins, amount):

    dp = [float('inf')] * (amount + 1)

    dp[0] = 0

    for i in range(1, amount + 1):

        for coin in coins:

            if coin <= i:
                dp[i] = min(dp[i], dp[i - coin] + 1)

    if dp[amount] == float('inf'):
        return -1

    return dp[amount]


coins = [1, 2, 5]
amount = 11
```

```
print("Minimum coins =", coin_change(coins, amount))
```

Output:

Minimum coins = 3

Because:

11 = 5 + 5 + 1

Complexity: O(n × amount)

---

2. 0/1 Knapsack

Rule: Each item can be taken **only once**.

```
def knapsack(weights, profits, capacity):

    n = len(weights)

    dp = [[0 for _ in range(capacity + 1)]
          for _ in range(n + 1)]

    for i in range(1, n + 1):

        for w in range(1, capacity + 1):

            if weights[i - 1] <= w:

                dp[i][w] = max(
                    dp[i - 1][w],
                    profits[i - 1] +
                    dp[i - 1][w - weights[i - 1]]
                )

            else:
                dp[i][w] = dp[i - 1][w]

    return dp[n][capacity]
```

```
weights = [3, 4, 6, 5]
profits = [2, 3, 1, 4]
capacity = 8

print("Maximum Profit =", knapsack(weights, profits, capacity))
```

Output:

Maximum Profit = 6

Selected items:

Weight = 3 + 5 = 8
Profit = 2 + 4 = 6

Complexity: O(n × W)

---

3. Matrix Chain Multiplication

Goal: Find the minimum number of scalar multiplications.

```
def matrix_chain(p):

    n = len(p) - 1

    dp = [[0 for _ in range(n)]
          for _ in range(n)]

    for length in range(2, n + 1):

        for i in range(n - length + 1):

            j = i + length - 1

            dp[i][j] = float('inf')

            for k in range(i, j):

                cost = (
                    dp[i][k]
                    + dp[k + 1][j]
                    + p[i] * p[k + 1] * p[j + 1]
```

```
            )

            dp[i][j] = min(dp[i][j], cost)

    return dp[0][n - 1]


# A = 10x20, B = 20x30, C = 30x40
p = [10, 20, 30, 40]

print("Minimum multiplications =", matrix_chain(p))
```

Output:

Minimum multiplications = 18000

Complexity: $O(n^3)$

---

4. LCS — Longest Common Subsequence

Example:

String 1 = abcde
String 2 = bdgek

LCS is:

bd

Practical Code

```
def lcs(X, Y):

    m = len(X)
    n = len(Y)

    dp = [[0 for _ in range(n + 1)]
          for _ in range(m + 1)]

    for i in range(1, m + 1):

        for j in range(1, n + 1):
```

```
            if X[i - 1] == Y[j - 1]:

                dp[i][j] = dp[i - 1][j - 1] + 1

            else:

                dp[i][j] = max(
                    dp[i - 1][j],
                    dp[i][j - 1]
                )

    # Find LCS string
    i = m
    j = n
    result = ""

    while i > 0 and j > 0:

        if X[i - 1] == Y[j - 1]:

            result = X[i - 1] + result
            i -= 1
            j -= 1

        elif dp[i - 1][j] > dp[i][j - 1]:

            i -= 1

        else:

            j -= 1

    return result, dp[m][n]


X = "abcde"
Y = "bdgek"

sequence, length = lcs(X, Y)

print("LCS =", sequence)
print("Length =", length)
```

Output:

LCS = bd
Length = 2

⭐ Practical Exam ke liye MOST IMPORTANT

Ye **14 programs** pakka practice karo:

Unit	Program	Main thing
I	Summation 3 methods	Loop, Formula, Recursion
I	Prime 3 methods	Factors
I	Fibonacci	Iterative + Recursive
I	Bubble Sort	Adjacent swap
I	Insertion Sort	Insert correct position
I	Selection Sort	Find minimum
II	Binary Search	Sorted array
II	Merge Sort	Divide + Merge
II	Quick Sort	Pivot
II	Karatsuba	Large multiplication
II	Strassen	7 multiplications
II	Min-Max	Divide & Conquer
II	Maximum Subarray	Maximum sum
III	Coin Change	DP
III	0/1 Knapsack	DP table
III	Matrix Chain	Minimum multiplication

III LCS Common subsequence

🧠 One-line tricks

Bubble: `Compare + Swap`
Insertion: `Take + Shift + Insert`
Selection: `Find Minimum + Swap`
Binary: `Middle + Half`
Merge: `Divide + Merge`
Quick: `Pivot + Partition`
Karatsuba: `3 multiplications instead of 4`
Strassen: `7 multiplications`
DP: `Store previous answers`
Knapsack: `Take or Don't Take`
MCM: `Best parenthesis`
LCS: `Common sequence, not necessarily continuous`